

Operating Systems Laboratory

B.Tech. 5th Semester



Name : Chandan Kumar
Roll Number : 22ETCS002301
Programme : B.Tech in CSE
Department : Computer Science and Engineering

Faculty of Engineering & Technology
Ramaiah University of Applied Sciences



Ramaiah University of Applied Sciences

Private University Established in Karnataka State by Act No. 15 of 2013

Faculty	Engineering & Technology
Programme	B. Tech. in Computer Science and Engineering
Year/Semester	3 rd year / 5 th Semester
Name of the Laboratory	Operating Systems Laboratory
Laboratory Code	CSL303A

List of Experiments

1. File management system calls
2. Process management system calls
3. Multithreaded programming
4. Round Robin Scheduling Algorithm
5. Priority Scheduling Algorithm
6. Dining philosopher problem using Semaphore
7. Readers Writers Problem using Semaphore

Index Sheet

No.	Lab Experiment	Lab Documents	Viva
1	Programs using file management system calls	10% of Lab CE	
2	Programs using process management system calls	10% of Lab CE	
3	Programs based on multithreaded programming	10% of Lab CE	
4	Round Robin Scheduling Algorithm Implementation	10% of Lab CE	
5	Priority Scheduling Algorithm	10% of Lab CE	
6	Solution to Dining Philosopher problem using Semaphore	10% of Lab CE	
7	Solution to Readers Writers Problem	10% of Lab CE	
8	Lab Internal Test	30% of Lab CE	

S. No	Experiments Name	Page No
1	File management system calls	5-7
2	Process management system calls	8-11
3	Multithreaded programming	12-17
4	Round Robin Scheduling Algorithm	18-21
5	Priority Scheduling Algorithm	22-25
6	Dining philosopher problem using Semaphore	26-30
7	Readers Writers Problem using Semaphore	31-35

Laboratory 1

Title of the Laboratory Exercise: Programs using file management system calls

1. Introduction and Purpose of Experiment

A system call is a programmatic way in which a computer program requests a service from the kernel of the operating system it is executed on. There are different types of system calls developed for various purposes. They are mainly classified as process management, file management, directory management. By solving the problems students will be able to apply file management system calls

Aim and Objectives

Aim

- To develop programs involving file management system calls

Objectives

At the end of this lab, the student will be able to

- Use different file management system calls
- Apply different system calls wherever required
- Create C programs using file management system calls

2. Experimental Procedure

- i. Analyse the problem statement
- ii. Design an algorithm for the given problem statement and develop a flowchart/pseudo-code
- iii. Implement the algorithm in C language
- iv. Compile the C program
- v. Test the implemented program
- vi. Document the Results
- vii. Analyse and discuss the outcomes of your experiment

3. Questions

Implement the following command in C

Implement copy command (cp) to copy a file content to other file using file management system calls

4. Calculations/Computations/Algorithms

Start

- Include necessary header files (stdio.h, unistd.h, fcntl.h).

Declare Buffers and File Descriptors

- Declare a buffer buf[] for reading and writing data.
- Declare file descriptors fd1 and fd2 for the two files.

Open the Files

- Open the first file file1.txt in read-only mode (O_RDONLY) and store the file descriptor in fd1.
- Open the second file file2.txt in write-only mode (O_WRONLY) and store the file descriptor in fd2.
- If fd1 == -1, print an error message and exit the program (return 1).

Copy Content from fd1 to fd2

- Use a while loop to read one byte at a time from fd1 using read(fd1, buf, 1) and write it to fd2 using write(fd2, buf, 1).
- Repeat until the end of the file is reached (when read returns 0).

Print Success Message

- Once the copy is complete, print "successful".

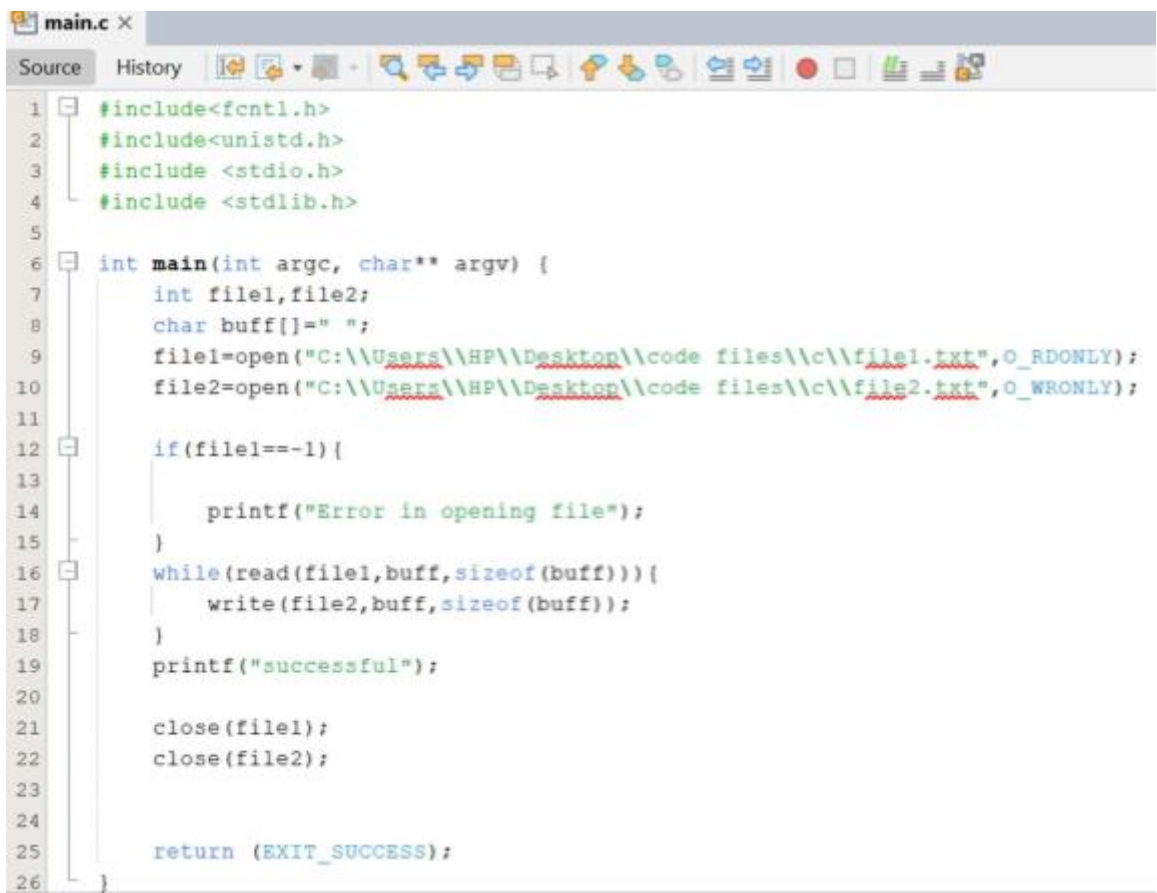
Close the Files

- Close both files using close(fd1) and close(fd2).

End

- Return 0 to indicate successful execution.

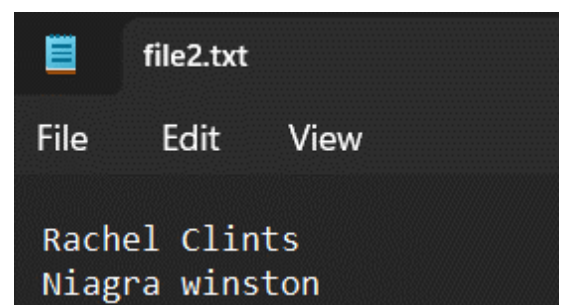
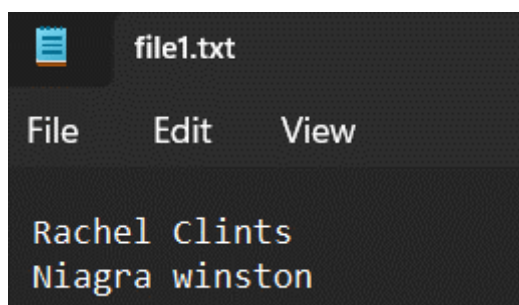
5. Presentation of Results



```
1 #include<fcntl.h>
2 #include<unistd.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main(int argc, char** argv) {
7     int file1,file2;
8     char buff[]=" ";
9     file1=open("C:\\Users\\HP\\Desktop\\code files\\c\\file1.txt",O_RDONLY);
10    file2=open("C:\\Users\\HP\\Desktop\\code files\\c\\file2.txt",O_WRONLY);
11
12    if(file1==-1){
13        printf("Error in opening file");
14    }
15    while(read(file1,buff,sizeof(buff))){
16        write(file2,buff,sizeof(buff));
17    }
18    printf("successful");
19
20    close(file1);
21    close(file2);
22
23
24
25    return (EXIT_SUCCESS);
26 }
```

Output :-

```
successful
RUN SUCCESSFUL (total time: 3s)
```



6. Analysis and Discussions

The program replicates the contents of `file1.txt` to `file2.txt` by using the file handling routines in C. It examines to see if `file1.txt` can be opened successfully to show error handling. But it doesn't have error handling while opening {file2.txt`. Furthermore, the method reads data using a fixed-size buffer, which might not be ideal for huge files.

Laboratory 2

Title of the Laboratory Exercise: Programs using process management system calls

2. Introduction and Purpose of Experiment

A system call is a programmatic way in which a computer program requests a service from the kernel of the operating system it is executed on. There are different types of system calls developed for various purposes. They are mainly classified as process management, file management, directory management. By solving the problems students will be able to apply process management system calls

Aim and Objectives

Aim

- To develop programs involving process management system calls

Objectives

At the end of this lab, the student will be able to

- Use different process management system calls
- Apply different system calls wherever required
- Create C programs using process management system calls

3. Experimental Procedure

- i. Analyse the problem statement
- ii. Design an algorithm for the given problem statement and develop a flowchart/pseudo-code
- iii. Implement the algorithm in C language
- iv. Compile the C program
- v. Test the implemented program
- vi. Document the Results
- vii. Analyse and discuss the outcomes of your experiment

4. Questions

Implement the following operations in C

Create four different processes (with different process ID) and assign four different tasks (addition, subtraction, Multiplication, division) to each process. All processes should display the result along with its process ID and parent process ID.

5. Calculations/Computations/Algorithms

Start

- Include the necessary header files (stdio.h, stdlib.h).

Declare Variables

- Declare integers a, b, c for process IDs and num1=20, num2=10 for the arithmetic operations.
- Declare a pointer wstatus to store the status of child processes.

Create First Child Process (Fork)

- Use fork() to create the first child process and store the return value in a.
- If a == 0 (Child Process):
 - Print the parent process ID using getppid() and the current child's process ID using getpid().
 - Perform the addition of num1 + num2 and print the result.
 - **Create a Second Child (Child1's First Child)**
 - Use fork() to create another child process, storing the return value in b.
 - If b == 0 (Child1's first child):
 - Print the process IDs and perform subtraction (num1 - num2), printing the result.
 - **Create a Third Child (Child1's Second Child)**
 - Use fork() to create another child, storing the return value in d.
 - If d == 0 (Child1's second child):
 - Print the process IDs and perform multiplication (num1 * num2), printing the result.

- Else, use waitpid() to wait for Child1 (b) to complete.
- Else (Parent Process):
 - **Create Second Child Process (Fork)**
 - Use fork() to create the second child and store the return value in c.
 - If c == 0 (Child2):
 - Print the process IDs and perform division (num1 / num2), printing the result.
 - Else, use waitpid() to wait for the first child (a) to complete.

End

- The program ends successfully using EXIT_SUCCESS.

6. Presentation of Results

```

main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  int main(int argc, char** argv) {
4      int a,b,c,num1=20,num2=10;
5      void *wstatus;
6      a=fork();
7      if(a==0)
8      {
9          printf("Parent PPID : %d\n",getppid());
10         printf("\nParent child1 ID is : %d\t Parent is :%d\n",getpid(),getppid());
11         printf(" Addition is %d\n", (num1+num2));
12         b=fork();
13         if(b==0)
14         {
15             printf("\nchild1s first child ID is : %d\t Parent is %d\n",getpid(),getppid());
16             printf(" Subtraction is %d\n", (num1-num2));
17         }
18         else
19         {
20             int d=fork();
21             if(d==0)
22             {
23                 printf("\nchild1s second child ID is : %d\t Parent is %d\n",getpid(),getppid());
24                 printf(" Multiplication is %d\n", (num1*num2));
25             }
26             else
27             {
28                 waitpid(b,&wstatus,0);
29             }
30         }
31     }
32     else
33     {
34         c=fork();
35         if(c==0)
36         {
37             printf("\nParent Child2 ID is : %d\t Parent is %d\n",getpid(),getppid());
38             printf(" Division is %d\n", (num1/num2));
39         }
40         else
41         {
42             waitpid(a,&wstatus,NULL);
43         }
44     }
45     return (EXIT_SUCCESS);
46 }

```

Output :-

```
Parent Child2 ID is : 16889      Parent is 16887
Division is 2
Parent PPID : 16887

Parent child1 ID is : 16888      Parent is :16887
Addition is 30

child1s first child ID is : 16890      Parent is 16888
Subtraction is 10
Parent PPID : 16887

Parent child1 ID is : 16888      Parent is :16887
Addition is 30

child1s second child ID is : 16891      Parent is 16888
Multiplication is 200
Parent PPID : 16887

Parent child1 ID is : 16888      Parent is :16887
Addition is 30
```

7. Analysis and Discussions

This program illustrates how to create and manage multiple processes using `fork()`. It assigns different arithmetic tasks to child processes and waits for their completion with `waitpid()`. The use of `fork()` creates a process tree where the parent process spawns children that execute specific tasks. The program effectively demonstrates inter-process communication and synchronization in a simple manner. However, care should be taken with process termination and resource management in more complex scenarios.

Laboratory 3

Title of the Laboratory Exercise: Programs based on multithreaded programming

1. Introduction and Purpose of Experiment

Multithreading is the ability of a processor or a single core in a multi-core processor to execute multiple threads concurrently, supported by the operating system. By solving students will be able to manipulate multiple threads in a program.

2. Aim and Objectives

Aim

- To develop programs using multiple threads.

Objectives

At the end of this lab, the student will be able to

- Identify multiple tasks
- Use threads constructs for creating threads
- Apply threads for different/multiple tasks

3. Experimental Procedure

- i. Analyse the problem statement
- ii. Design an algorithm for the given problem statement and develop a flowchart/pseudo-code
- iii. Implement the algorithm in C language
- iv. Compile the C program
- v. Test the implemented program
- vi. Document the Results
- vii. Analyse and discuss the outcomes of your experiment

4. Questions

Create multithreaded programs to implement the following

- a) Display "Hello World" message by 3 different threads
- b) Create two threads;
 - Thread1 adds marks (out of 10) of student1 from subject1 to subject5, Thread2 adds marks of student2 from subject1 to subject 5. Main process takes the sum returned from the Thread1 and Thread2, decides who scored more marks and displays student with its highest score.

5. Calculations/Computations/Algorithms

a.) Algorithm for Part A

Start

- Include the necessary header files: `stdio.h`, `stdlib.h`, and `pthread.h`.

Define the Thread Function (fun)

- Input: A void pointer `arg`.
- Print "Hello World".
- Exit the thread using `pthread_exit(NULL)`.

Inside the main Function

- Declare an array of thread identifiers `t_id[3]` to store the thread IDs of three threads.
- Initialize a loop variable `i`.

Create Threads in a Loop

- Use a for loop to iterate three times (`i = 0` to `2`).
- Inside the loop:
 - Call `pthread_create()` to create a new thread, passing:
 - `&t_id[i]` as the pointer to store the thread ID.
 - `NULL` for default attributes.
 - `fun` as the thread function to execute.
 - `NULL` as the argument for the thread function.
 - Store the return value of `pthread_create()` in `x`.

Check Thread Creation Status

- If `x == 0` (thread creation succeeded):

- Print: "Thread {i+1} created successfully".
- Else (thread creation failed):
 - Print: "Failed to create thread".
- Print the thread ID (t_id[i]).

End the Program

- Return EXIT_SUCCESS to indicate successful program execution.

b.) Algorithm for Part B**Start**

Initialize the program and include required header files (stdio.h, stdlib.h, and pthread.h).

Define the Thread Function (fun)

- Input: A void pointer arg (assumed to point to an integer array).
- Convert arg to an integer pointer.
- Initialize a variable sum to store the total marks.
- Use a for loop to iterate over the first 5 elements of the array, adding each element to sum.
- Return the sum as the result.

Inside the Main Function (main)

- Define arrays stu1 and stu2 containing marks of two students.
- Declare an integer array result[2] to store the sums of marks from two threads.
- Declare two thread IDs, tid1 and tid2.

Create and Execute the First Thread

- Use pthread_create() to create a thread (tid1) that executes the fun function with stu1 as input.
- Wait for the thread to finish execution using pthread_join() and store its return value in result[0].

Create and Execute the Second Thread

- Use `pthread_create()` to create a second thread (`tid2`) that executes the `fun` function with `stu2` as input.
- Wait for the second thread to finish execution using `pthread_join()` and store its return value in `result[1]`.

Compare the Results

- If `result[0]` (sum of marks for student 1) is greater than `result[1]` (sum of marks for student 2):
 - Print: "student1 has highest marks".
- Else if `result[1]` is greater than `result[0]`:
 - Print: "student2 has highest marks".
- Otherwise (if both results are equal):
 - Exit the thread using `pthread_exit()`.

End the Program

- Return `EXIT_SUCCESS` to indicate the program executed successfully.

6. Presentation of Results

a.)

```
main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  void* fun(void *arg)
4  {
5
6      printf("Hello World\n");
7
8      pthread_exit(NULL);
9  }
10 int main(int argc, char** argv) {
11     pthread_t t_id[3];
12
13     int i=0;
14     for(i=0;i<3;i++)
15     {
16         int x=pthread_create(&t_id[i],NULL,fun,NULL);
17         if(x==0)
18         {
19             printf("Thread %d created successfully\n",(i+1));
20
21         }
22         else
23         {
24             printf("Failed to created thread\n");
25         }
26         printf("Thread id is %d\n",t_id[i]);
27     }
28
29     return (EXIT_SUCCESS);
30 }
```

Output:

Thread 1 created successfully
Thread id is -526174528
Thread 2 created successfully
Thread id is -536664384
Thread 3 created successfully
Thread id is -547154240

b.)

```
main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  void* fun(void *arg)
5  {
6      int *marks=(int*)arg;
7      int sum=0,i;
8      for(i=0;i<=4;i++)
9      {
10         sum=sum+ marks[i];
11     }
12 }
13
14 int main(int argc, char** argv) {
15
16     int stu1[5]={50,60,39,69,70};
17     int stu2[5]={80,66,59,99,60};
18     int result[2];
19     pthread_t tid1,tid2;
20     result[0]=pthread_create(&tid1,NULL,fun,stu1);
21     pthread_join(tid1,(void*)result[0]);
22     result[1]= pthread_create(&tid1,NULL,fun,stu2);
23     pthread_join(tid2,(void*)result[1]);
24     if(result[0]>result[1])
25     {
26         printf("stu1 has highest marks");
27     }
28     else if(result[1]>result[0])
29     {
30         printf("stu2 has highest marks");
31     }
32     else
33     {
34         pthread_exit(NULL);
35     }
36     return (EXIT_SUCCESS);
37 }
38
39
40
41
42
```

Output:

stu2 has highest marks

7. Analysis and Discussions

The first program focuses on the basics of thread creation and execution by printing "Hello World" from multiple threads. It demonstrates thread initialization in a loop, ensuring efficient handling of repetitive tasks. The program checks for successful thread creation, though it lacks proper synchronization (`pthread_join()`), which could lead to premature termination of threads. Despite its simplicity, it serves as an excellent introduction to multithreading.

The second program demonstrates the use of multithreading to calculate the total marks of two students and determine who has the highest score. It utilizes a thread function to perform the summation in parallel for each student, showcasing the efficiency of concurrent execution. Threads are managed using `pthread_create()` and `pthread_join()`, with results passed back to the main thread for comparison. While this approach effectively illustrates thread usage for independent computations, it lacks dynamic array handling and robust error checks.

Both programs highlight fundamental multithreading concepts, with the first emphasizing computational tasks and result retrieval, and the second focusing on thread creation mechanics. These examples provide a foundation for building more complex multithreaded applications.

Laboratory 4

Title of the Laboratory Exercise: Round Robin Scheduling Algorithm Implementation

1. Introduction and Purpose of Experiment

The Round Robin (RR) scheduling algorithm allocates fixed time slices (time quantum) to processes in a cyclic order. Each process gets equal CPU time, ensuring fair allocation. This experiment helps students understand process scheduling and CPU time sharing.

2. Aim and Objectives

Aim

- To implement and demonstrate the Round Robin Scheduling algorithm for processes with an arrival time of 0.

Objectives

At the end of this lab, the student will be able to

- Implement the Round Robin Scheduling algorithm.
- Calculate turnaround time and waiting time for each process.
- Determine average waiting and turnaround times.
- Analyse the effect of time quantum on process execution.

3. Experimental Procedure

- i. Analyse the problem statement
- ii. Design an algorithm for the given problem statement and develop a flowchart/pseudo-code
- iii. Implement the algorithm in C language
- iv. Compile the C program
- v. Test the implemented program
- vi. Document the Results
- vii. Analyse and discuss the outcomes of your experiment

4. Questions

Write program to simulate the following process scheduling algorithms. Calculate average waiting time and average turnaround time for processes under each scheduling algorithm.

Instructions: Assume all the processes arrive at the same time. For round robin scheduling algorithm, read the number of processes in the system, their CPU burst times and the size of the time slice.

- a) Round Robin

5. Calculations/Computations/Algorithms

Input:

- Read the number of processes NNN , their burst times $BT[]$, and the time quantum $TQTQTQ$.

Initialization:

- Set $Temp[] = BT[]$ (to track remaining burst times).
- Initialize time = 0 and remain = N .

Execution Loop:

- While there are unfinished processes:
 - For each process:
 - If remaining burst time $Temp[i] \leq TQTQTQ$:
 - Execute the process completely.
 - Update time and calculate:
 - Turnaround Time ($TAT[i]$).
 - Waiting Time ($WT[i]$).
 - Decrease the number of remaining processes.
 - If $Temp[i] > TQTQTQ$:
 - Execute the process for $TQTQTQ$ and reduce $Temp[i]$.

Output:

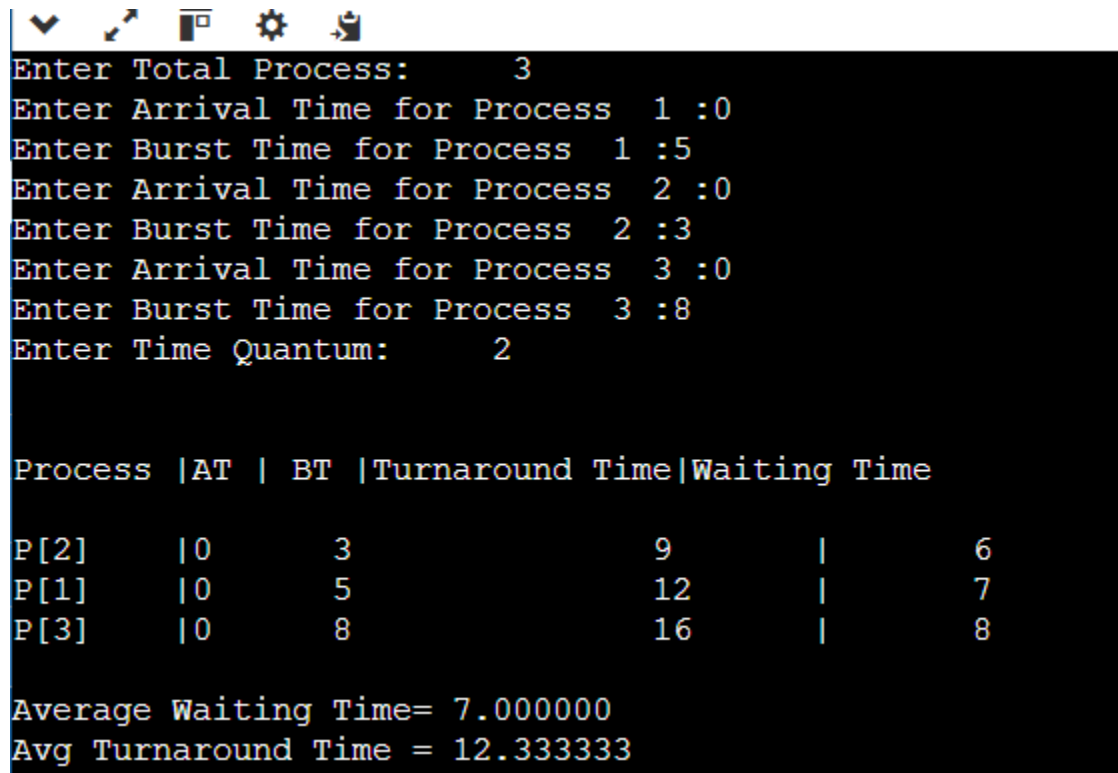
- Display AT[i],BT[i],TAT[i],WT[i]AT[i], BT[i], TAT[i], WT[i]AT[i],BT[i],TAT[i],WT[i] for each process.
- Calculate and print:
 - Average Waiting Time.
 - Average Turnaround Time.

6. Presentation of Results

```

main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  int main(int argc, char** argv) {
4  int i,j,N,time,remain,count=0,TQ,TAT,WT;
5  int Sum_wt=0, Sum_tat=0, AT[10], BT[10], Temp[10];
6  printf("Enter Total Process:\t ");
7  scanf("%d",&N);
8  remain=N;
9  for(i=0;i<N;i++)
10 {
11     printf("Enter Arrival Time for Process  %d :",i+1);
12     scanf("%d",&AT[i]);
13     printf("Enter Burst Time for Process  %d :",i+1);
14     scanf("%d",&BT[i]);
15     Temp[i]=BT[i];
16 }
17 printf("Enter Time Quantum:\t");
18 scanf("%d",&TQ);
19 printf("\n\nProcess\t|AT | BT |Turnaround Time|Waiting Time\n\n");
20 for(time=0,i=0;remain!=0;)
21 {
22     if(Temp[i]<=TQ && Temp[i]>0)
23     {
24         time=time+Temp[i];
25         Temp[i]=0;
26         count=1;
27     }
28     else if(Temp[i]>0)
29     {
30         Temp[i]=Temp[i]-TQ;
31         time=time+TQ;
32     }
33     if(Temp[i]==0 && count==1)
34     {
35         remain--;
36         TAT=time-AT[i];
37         WT=TAT-BT[i];
38         printf("P[%d]\t| %d\t%d\t%d\t%d\t| \t%d\n",i+1,AT[i],BT[i],TAT,WT);
39         Sum_wt=Sum_wt+WT;
40         Sum_tat=Sum_tat+TAT;
41         count=0;
42     }
43     if(i==N-1)
44         i=0;
45     else if(AT[i+1]<=time)
46         i++;
47     else
48         i=0;
49 }
50 printf("\nAverage Waiting Time= %f\n", (Sum_wt*1.0)/N);
51 printf("Avg Turnaround Time = %f\n", (Sum_tat*1.0)/N);
52
53     return (EXIT_SUCCESS);
54 }
55

```

OUTPUT :-


```

Enter Total Process:      3
Enter Arrival Time for Process 1 :0
Enter Burst Time for Process 1 :5
Enter Arrival Time for Process 2 :0
Enter Burst Time for Process 2 :3
Enter Arrival Time for Process 3 :0
Enter Burst Time for Process 3 :8
Enter Time Quantum:      2

Process |AT | BT |Turnaround Time|Waiting Time
P[2]    |0   3      9      |      6
P[1]    |0   5     12      |      7
P[3]    |0   8     16      |      8

Average Waiting Time= 7.000000
Avg Turnaround Time = 12.333333

```

7. Analysis and Discussions

In Round Robin Scheduling, processes are scheduled in a circular manner with a fixed time quantum. This approach ensures fairness among processes and prevents starvation, making it suitable for time-sharing systems. Nonetheless, its fixed time quantum assumption may not be optimal for all scenarios, and frequent context switches can introduce overhead.

Laboratory 5

Title of the Laboratory Exercise: Program for Priority Scheduling Algorithm

1. Introduction and Purpose of Experiment

A Process Scheduler schedules different processes to CPU based on particular scheduling algorithms. There are various scheduling algorithms present in each group of operating system. By solving these problems students will be able use different scheduling algorithms as part of their implementation

2. Aim and Objectives

Aim

- To develop program to implement Priority Scheduling Algorithm

Objectives

At the end of this lab, the student will be able to

- Implement the Priority Scheduling Algorithm.
- Calculate turnaround time and waiting time for each process.
- Determine average waiting and turnaround times.
- Analyse the effect of priority on process execution.

3. Experimental Procedure

- i. Analyse the problem statement
- ii. Design an algorithm for the given problem statement and develop a flowchart/pseudo-code
- iii. Implement the algorithm in C language
- iv. Compile the C program
- v. Test the implemented program
- vi. Document the Results
- vii. Analyse and discuss the outcomes of your experiment

4. Questions

Write program to simulate the Priority scheduling algorithms. Calculate average waiting time and average turnaround time for processes under scheduling algorithm.

Instructions: Assume all the processes arrive at the same time. For priority scheduling algorithm, read the number of processes in the system, their CPU burst times and the priorities.

a) Priority

5. Calculations/Computations/Algorithms

Priority Scheduling Algorithm:

1. Input Process Details:

- Gather the number of processes.
- For each process, record its CPU burst time and priority.

2. Sort Processes by Priority:

- Arrange all processes in ascending order of their priority values (lower priority number means higher priority).
- If two processes have the same priority, keep them in the order they were entered (non-preemptive).

3. Initialize Time Tracking:

- Start with a total elapsed time of 0.
- Waiting times and turnaround times for each process are initially set to 0.

4. Calculate Waiting Times:

- For each process:
 - The waiting time is the total time elapsed before the process starts execution.
 - Keep a running total of the time taken by previously executed processes.

5. Calculate Turnaround Times:

- For each process:
 - The turnaround time is the total time from when the process arrived until it finishes execution.
 - This can be determined by adding the burst time of the current process to its waiting time.

6. Compute Averages:

- Track the cumulative waiting times and turnaround times as processes complete.
- At the end, divide these cumulative values by the total number of processes to get the averages.

7. Display Results:

- Output the processes in the order of execution along with their burst times, priorities, waiting times, and turnaround times.
- Also display the average waiting time and average turnaround time.

6. Presentation of Results

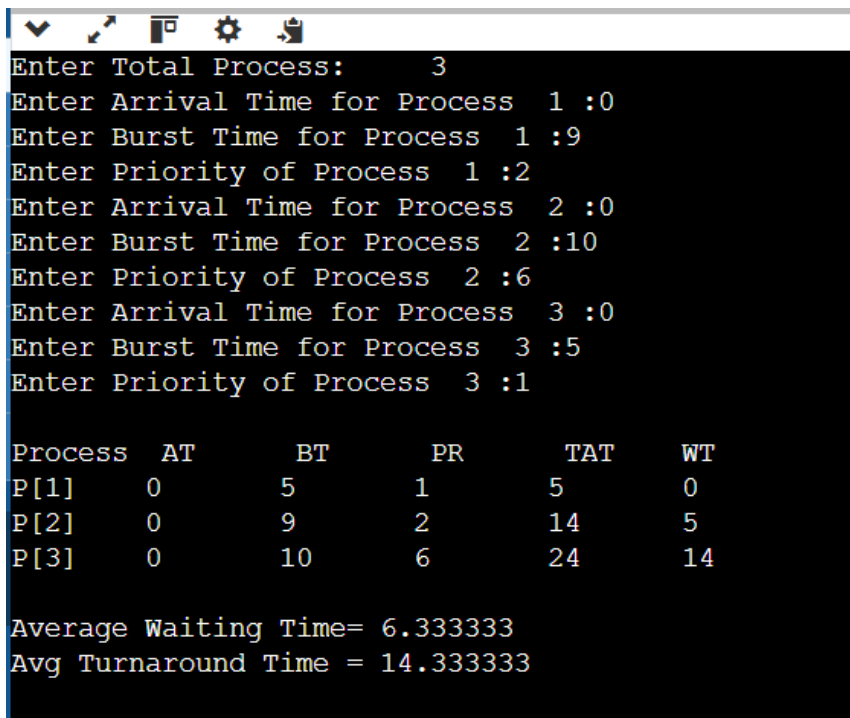
Code :-

```

main.c
1 //Priority Scheduling Algorithm
2
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main(int argc, char** argv) {
7     int i,j,N,TAT[10],WT[10],pos,temp;
8     int Sum_wt=0,Sum_tat=0,AT[10],BT[10],PR[10];
9     printf("Enter Total Process:\t ");
10    scanf("%d",&N);
11
12    for(i=0;i<N;i++)
13    {
14        printf("Enter Arrival Time for Process %d :",i+1);
15        scanf("%d",&AT[i]);
16        printf("Enter Burst Time for Process %d :",i+1);
17        scanf("%d",&BT[i]);
18        printf("Enter Priority of Process %d :",i+1);
19        scanf("%d",&PR[i]);
20    }
21    //Sort the processes according to priority
22    for(i=0;i<N;i++)
23    {
24        pos=i;
25        for(j=i+1;j<N;j++)
26        {
27            if(PR[j]<PR[pos])
28            {
29                pos=j;
30            }
31        }
32        temp=PR[i];
33        PR[i]=PR[pos];
34        PR[pos]=temp;
35        temp=BT[i];
36        BT[i]=BT[pos];
37        BT[pos]=temp;
38    }
39    WT[0]=0;
40    printf("\nProcess\t AT\t BT \t PR\t TAT\tWT\n");
41    for(i=0;i<N;i++)
42    {
43        WT[i]=0;
44        TAT[i]=0;
45        for(j=0;j<i;j++)
46        {
47            WT[i]=WT[i]+BT[j];
48        }
49        TAT[i]=WT[i]+BT[i];
50        Sum_wt=Sum_wt+WT[i];
51        Sum_tat=Sum_tat+TAT[i];
52        printf("P[%d]\t%d\t%d\t%d\t%d\t%d\n",i+1,AT[i],BT[i],PR[i],TAT[i],WT[i]);
53    }
54    printf("\nAverage Waiting Time= %f\n", (Sum_wt*1.0)/N);
55    printf("Avg Turnaround Time = %f\n", (Sum_tat*1.0)/N);
56    return (EXIT_SUCCESS);
57 }

```


Output :-



```
Enter Total Process:      3
Enter Arrival Time for Process  1 :0
Enter Burst Time for Process  1 :9
Enter Priority of Process  1 :2
Enter Arrival Time for Process  2 :0
Enter Burst Time for Process  2 :10
Enter Priority of Process  2 :6
Enter Arrival Time for Process  3 :0
Enter Burst Time for Process  3 :5
Enter Priority of Process  3 :1

Process  AT      BT      PR      TAT      WT
P[1]     0        5        1        5        0
P[2]     0        9        2       14        5
P[3]     0       10        6       24       14

Average Waiting Time= 6.333333
Avg Turnaround Time = 14.333333
```

7. Analysis and Discussions

In Priority Scheduling, processes are scheduled based on their priority levels, with lower values indicating higher priority. The algorithm calculates turnaround time and waiting time for each process, providing insights into process scheduling efficiency. However, it lacks considerations for preemptive behavior and dynamic priority adjustments, which are essential for real-time systems.

Laboratory 6

Title of the Laboratory Exercise: Solution to Dining Philosopher problem using Semaphore

1. Introduction and Purpose of Experiment

In multitasking systems, simultaneous use of critical section by multiple processes leads to data inconsistency and several other concurrency issues. By solving this problem students will be able to use semaphore for synchronisation purpose in concurrent programs.

2. Aim and Objectives

Aim

- To develop concurrent programs using semaphores

Objectives

At the end of this lab, the student will be able to

- Use semaphore
- Apply appropriate semaphores in different contexts
- Develop concurrent programs using semaphores

3. Experimental Procedure

- Analyse the problem statement
- Design an algorithm for the given problem statement and develop a flowchart/pseudo-code
- Implement the algorithm in C language
- Compile the C program
- Test the implemented program
- Document the Results
- Analyse and discuss the outcomes of your experiment

4. Question

Implement the Dining Philosopher problem using POSIX threads

5. Calculations/Computations/Algorithms

Algorithm

1. Initialize Variables:

- Define the number of philosophers (N).
- Create arrays to hold the state of each philosopher (state) and their thread IDs (thread_id).
- Define semaphores:
 - mutex: For critical section access.
 - S[N]: For each philosopher to control their access to forks.

2. Define States:

- Define constants for the states: THINKING, HUNGRY, and EATING.

3. Calculate Neighbor Indices:

- Define macros or functions to calculate the indices of the left and right neighbors:
 - $LEFT = (phnum + N - 1) \% N$
 - $RIGHT = (phnum + 1) \% N$

4. Implement Helper Functions:

- **test(phnum):**
 - If the philosopher is HUNGRY and neither neighbor is EATING, change the state to EATING and signal the philosopher's semaphore.
- **take_fork(phnum):**
 - Acquire the mutex lock.
 - Set the philosopher's state to HUNGRY and print a message.
 - Call test(phnum) to check if they can start eating.
 - Release the mutex lock.
 - Wait on the philosopher's semaphore (S[phnum]).
- **put_fork(phnum):**
 - Acquire the mutex lock.
 - Set the philosopher's state to THINKING and print a message.
 - Call test(LEFT) and test(RIGHT) to check if neighbors can eat.

- Release the mutex lock.

5. Philosopher Thread Function:

- Implement a function that simulates a philosopher:
 - Loop infinitely, alternating between thinking, taking forks, eating, and putting forks down.

6. Main Function:

- Initialize the mutex semaphore.
- Initialize each philosopher's semaphore (S[i]).
- Create a thread for each philosopher using pthread_create.
- Join all philosopher threads using pthread_join.

7. Synchronization:

- Use semaphores to enforce mutual exclusion and ensure no deadlock or starvation.

6. Presentation of Results

```
main.c
1  #include <stdio.h>
2  #include <semaphore.h>
3  #include <pthread.h>
4  #include <unistd.h>
5
6  #define N 5          // Number of philosophers
7
8  #define HUNGRY 0
9  #define THINKING 1
10 #define EATING 2
11 #define LEFT (phnum + N - 1) % N // Left philosopher
12 #define RIGHT (phnum + 1) % N    // Right philosopher
13
14 int state[N];          // State of philosophers
15 int phil[N];          // Philosopher IDs
16
17 sem_t mutex;          // Semaphore to protect critical sections
18 sem_t s[N];           // Semaphore for each philosopher
19
20 void test(int phnum) {
21     if (state[phnum] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
22         state[phnum] = EATING;
23         printf("Philosopher %d takes forks %d and %d\n", phnum + 1, LEFT + 1, RIGHT + 1);
24         printf("Philosopher %d is eating\n", phnum + 1);
25         sleep(1); // Simulate eating
26         sem_post(&s[phnum]); // Allow philosopher to proceed
27     }
28 }
29
30 void take_fork(int phnum) {
31     sem_wait(&mutex); // Enter critical section
32     state[phnum] = HUNGRY;
33     printf("Philosopher %d is Hungry\n", phnum + 1);
34     test(phnum); // Try to acquire forks
35     sem_post(&mutex); // Exit critical section
36     sem_wait(&s[phnum]); // Wait until forks are available
37 }
```

```

39 void put_fork(int phnum) {
40     sem_wait(&mutex);          // Enter critical section
41     state[phnum] = THINKING;
42     printf("Philosopher %d puts down forks %d and %d\n", phnum + 1, LEFT + 1, RIGHT + 1);
43     printf("Philosopher %d is thinking\n", phnum + 1);
44     test(LEFT);                // Notify left philosopher
45     test(RIGHT);               // Notify right philosopher
46     sem_post(&mutex);          // Exit critical section
47 }
48
49 void *philosopher(void *num) {
50     int *phnum = (int *)num;
51     sleep(1);                  // Thinking
52     take_fork(*phnum);          // Try to eat
53     sleep(1);                  // Eating
54     put_fork(*phnum);           // Release forks
55     return NULL;
56 }
57
58 int main() {
59     pthread_t thread_id[N];
60
61     sem_init(&mutex, 0, 1);     // Initialize mutex semaphore
62     for (int i = 0; i < N; i++) {
63         sem_init(&s[i], 0, 0); // Initialize semaphores for each philosopher
64         phil[i] = i;           // Set philosopher IDs
65         state[i] = THINKING;   // Initialize states
66     }
67
68     // Create philosopher threads
69     for (int i = 0; i < N; i++) {
70         pthread_create(&thread_id[i], NULL, philosopher, &phil[i]);
71     }
72
73     // Wait for all threads to finish
74     for (int i = 0; i < N; i++) {
75         pthread_join(thread_id[i], NULL);
76     }
77
78     return 0;
79 }

```

Output :-

```

Philosopher 3 is Hungry
Philosopher 3 takes forks 2 and 4
Philosopher 3 is eating
Philosopher 4 is Hungry
Philosopher 5 is Hungry
Philosopher 5 takes forks 4 and 1
Philosopher 5 is eating
Philosopher 2 is Hungry
Philosopher 1 is Hungry
Philosopher 3 puts down forks 2 and 4
Philosopher 3 is thinking
Philosopher 2 takes forks 1 and 3
Philosopher 2 is eating
Philosopher 5 puts down forks 4 and 1
Philosopher 5 is thinking
Philosopher 4 takes forks 3 and 5
Philosopher 4 is eating
Philosopher 2 puts down forks 1 and 3
Philosopher 2 is thinking
Philosopher 1 takes forks 5 and 2
Philosopher 1 is eating
Philosopher 4 puts down forks 3 and 5
Philosopher 4 is thinking
Philosopher 1 puts down forks 5 and 2
Philosopher 1 is thinking

```

7. Analysis and Discussions

The code efficiently uses mutex and semaphores for synchronization to solve the dining philosopher's problem. In order to avoid malnutrition and impasse, it makes sure philosophers can only eat when both forks are accessible. It does not, however, address questions of fairness that might influence philosophers' access to forks in a particular order. All in all, it offers a concise overview of concurrent programming synchronization methods.

Laboratory 7

Title of the Laboratory Exercise: Solution to Readers Writers Problem using Semaphore

1. Introduction and Purpose of Experiment

In multitasking systems, simultaneous use of critical section by multiple processes leads to data inconsistency and several other concurrency issues. By solving this problem students will be able to use semaphore for synchronisation purpose in concurrent programs.

2. Aim and Objectives

Aim

- To develop concurrent programs using semaphores

Objectives

At the end of this lab, the student will be able to

- Use semaphore
- Apply appropriate semaphores in different contexts
- Develop concurrent programs using semaphores

3. Experimental Procedure

- i. Analyse the problem statement
- ii. Design an algorithm for the given problem statement and develop a flowchart/pseudo-code
- iii. Implement the algorithm in C language
- iv. Compile the C program
- v. Test the implemented program
- vi. Document the Results
- vii. Analyse and discuss the outcomes of your experiment

4. Question

Implement the Readers Writers problem using POSIX threads

5. Calculations/Computations/Algorithms

Algorithm**Initialization**

1. Create and initialize:
 - A semaphore wrt initialized to 1 (controls writer access).
 - A mutex mutex for protecting critical sections involving readcount.
 - A shared variable cnt (the data to be read or written).
 - A readcount variable initialized to 0 (keeps track of active readers).
-

Writer Thread

1. Wait on the semaphore wrt to acquire exclusive access to the shared resource.
 2. Modify the shared resource (cnt).
 3. Signal the semaphore wrt to release the lock on the shared resource.
-

Reader Thread

1. Lock the mutex to enter the critical section to modify readcount.
 2. Increment readcount:
 - If this is the first reader (readcount == 1), wait on the semaphore wrt to block writers.
 3. Unlock the mutex.
 4. Read the shared resource (cnt).
 5. Lock the mutex again to decrement readcount:
 - If this is the last reader (readcount == 0), signal the semaphore wrt to allow writers.
 6. Unlock the mutex.
-

Main Function

1. Initialize mutex and wrt.
2. Create reader threads and writer threads.
3. Wait for all threads to complete using pthread_join.
4. Destroy the mutex and semaphore.

Step-by-Step Algorithm

1. Initialization:

- `sem_init(&wrt, 0, 1)`
- `pthread_mutex_init(&mutex)`

2. Writer Process:

- Wait: `sem_wait(&wrt)`
- Critical Section:
 - Modify the shared variable (cnt).
- Signal: `sem_post(&wrt)`

3. Reader Process:

- Lock: `pthread_mutex_lock(&mutex)`
- Increment readcount:
 - If `readcount == 1`, wait: `sem_wait(&wrt)`
- Unlock: `pthread_mutex_unlock(&mutex)`
- Critical Section:
 - Read the shared variable (cnt).
- Lock: `pthread_mutex_lock(&mutex)`
- Decrement readcount:
 - If `readcount == 0`, signal: `sem_post(&wrt)`
- Unlock: `pthread_mutex_unlock(&mutex)`

4. Thread Execution in Main:

- Create n reader threads and m writer threads.
- Wait for all threads to complete.

5. Cleanup:

- Destroy the mutex and wrt semaphore.

6. Presentation of Results

```
main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <semaphore.h>
5  sem_t wrt;
6  pthread_mutex_t mutex;
```

```
7  int cnt = 1;
8  int readcount = 0;
9
10 void *writer(void *wno)
11 {
12     sem_wait(&wrt);
13     cnt = cnt*4;
14     printf("Writer %d modified cnt to %d\n",*((int *)wno),cnt);
15     sem_post(&wrt);
16 }
17
18 void *reader(void *rno)
19 {
20
21     pthread_mutex_lock(&mutex);
22     readcount++;
23     if(readcount == 1) {
24         sem_wait(&wrt);
25     }
26     pthread_mutex_unlock(&mutex);
27
28     printf("Reader %d: read cnt as %d\n",*((int *)rno),cnt);
29     pthread_mutex_lock(&mutex);
30     readcount--;
31     if(readcount== 0) {
32         sem_post(&wrt);
33     }
34     pthread_mutex_unlock(&mutex);
35 }
36
37 int main(int argc, char** argv) {
38     pthread_t read[10],write[5];
39     pthread_mutex_init(&mutex, NULL);
40     sem_init(&wrt,0,1);
41     int i;
42     int a[10] = {1,2,3,4,5,6,7,8,9,10};
43
44     for( i = 0; i < 10; i++) {
45         pthread_create(&read[i], NULL, (void *)reader, (void *)&a[i]);
46     }
47     for( i = 0; i < 5; i++) {
48         pthread_create(&write[i], NULL, (void *)writer, (void *)&a[i]);
49     }
50
51     for( i = 0; i < 10; i++) {
52         pthread_join(read[i], NULL);
53     }
54     for( i = 0; i < 5; i++) {
55         pthread_join(write[i], NULL);
56     }
57     pthread_mutex_destroy(&mutex);
58     sem_destroy(&wrt);
59     return (EXIT_SUCCESS);
60 }
```

Output :-

```
Reader 1: read cnt as 1
Reader 2: read cnt as 1
Reader 4: read cnt as 1
Reader 5: read cnt as 1
Reader 6: read cnt as 1
Reader 7: read cnt as 1
Reader 8: read cnt as 1
Reader 10: read cnt as 1
Writer 1 modified cnt to 4
Writer 2 modified cnt to 16
Writer 3 modified cnt to 64
Writer 4 modified cnt to 256
Writer 5 modified cnt to 1024
Reader 3: read cnt as 1024
Reader 9: read cnt as 1024
```

7. Analysis and Discussions

The code effectively demonstrates the readers-writers problem using semaphores and mutex for synchronization. Writers have exclusive access to the critical section while multiple readers can access it concurrently. However, fairness and potential starvation issues for writers may need further consideration for real-world scenarios.